Programming 2025/2026

Laboratory

Session 3 (April 21)

Functions

First Cycle Degree/Bachelor in Genomics
Dept. Of Pharmacy and Biotechnology (FaBiT)
University of Bologna

Functions

```
def function_name(parameters):
    expression
    ...
    expression
    return expression
```

- A function separates blocks of code from the main program
    - Defined once and can be used several times
- A function has a name
    - By using descriptive names, the program code will be easier to understand and debug
- A function may have a parameter or a set of parameters in **input**
- A function has a body
- A function may return one or more values as **output**

Function Definition vs Function Call

```
#function definition
def function_name(formal parameters):
    expression
    ...
    expression
    return expression


#function invocation/call
function_name(actual parameters)
```

1. The actual parameters are evaluated and the formal parameters are bound to the resulting values
2. The point of execution moves to the first statement of the function
3. The code in the body of the function is executed until either a **return** statement, or there are no more statements to execute
4. The value of the invocation is the returned value
5. The point of execution is transferred back to the code following immediately the invocation

Function Definition vs Function Call

```
#function definition
def mysum(x, y):
    print(x, y, x + y)

#function invocation/call
mysum(3, 4)
```

formal parameters

actual parameters

What is the return value of the function call `mysum(3, 4)`?

`None` value

- **`None`** is the value of a function which does not return any value
- Its type is **`NoneType`**
- What happens with the following code?

```
def f():
    x = 10


print(f())
```

- We can use **`None`** in an expression like:

```
M == None
M != None
```

Function Definition vs Function Call

```
#function definition
def mysum(x, y):
    return x + y


#function invocation/call
a = 3
b = 4
c = mysum(a, b)
```

formal parameters

actual parameters

What is the return value of the function call `mysum(a, b)`?
Why is it assigned to the variable `c`?

Function Call

```
#function definition
def mysum(x, y):
    return(x + y)

#function invocation/call
c = mysum(3, 4)
```

Positional method for parameter bound

Function Call

```
#function definition
def mysum(x, y):
    return(x + y)

#function invocation/call
c = mysum(y=4, x = 3)

# what happens with this call?
c = mysum(y=4, 3)
```

Keyword arguments method for parameter bound

Function Call

```
#function definition
def mysum(x, y=4):
    return(x + y)

#function invocation/call
c = mysum(3)
c = mysum(3,5)

# what happens with this function definition?
def mysum(x=3, y):
    return(x + y)
```

Default parameter values

import

```
import module_name
def function_name(parameters):
    expression
    …
    module_name.function_name()
    expression
    return expression
```

- Functions can be defined and stored in a separate file called *module* (e.g., **`random`**, **`string`**, or even your own)
  - Allows to hide the details of the code and focus on its functionality in the program
- The **`import`** keyword allows to import an external module and use the defined functions
  - A module must be imported before use
  - The function inside a module is called using "." added to the module name
- Online documentation: https://docs.python.org/3/library/

Comments

```
def function_name(parameters):
    '''comment describing the function'''
    expression
    ...
    expression  # short comment on this expression
    return expression
```

- Are used mainly for source code documentation
- Are ignored by the Python interpreter

Example

formal parameters

```
import math
def distance(a, b): #function definition
    c = a**2+b**2
    d = math.sqrt(c)
    return d
```

actual parameters

```
x = 3
y = 4
d = distance(x, y) #function call
print("The distance between (", x, ",", y, ") and the point
(0,0) is", d)
```

Example

```
import math
def distance(a, b):
    '''A function that
        - assumes two integer input a and b
        - returns (as a float value) the distance between (0,0)
and (a, b)'''
    c = a**2+b**2
    d = math.sqrt(c)  # function call: sqr root using a and b
    return d

x = 2
y = 3
d = distance(x, y) # function call: distance using x and y
print("The distance between (", x, ",", y, ") and the point
(0,0) is", d)
```

Call by Object Reference

```
#function definition
def function_name(formal parameters):
    expression
    ...
    expression
    return expression


#function invocation/call
function_name(actual parameters)
```

- Arguments are passed to a function using *call by object reference*
    - The value of an immutable object (i.e. numbers, Booleans, strings, tuples) passed as a parameter cannot be modified, any change within the *scope* of the function is done via the local variables and is NOT visible outside of the function
    - A change that a function makes to the value of a mutable object (i.e. lists, dictionaries, user defined objects) passed as a parameter is reflected outside the function
    - If we re-assign the parameter inside the function, the changes will NOT be reflected outside the function irrespective of whether the parameter is mutable or immutable

Example

- What happens with the following code?

```
def fun(n, st, l1, l2):
    n = n*2
    st = st.capitalize()
    l1.append(20)
    l2 = [10,20]
    print("Inside the function: ", n, st, l1, l2)

my_n = 5
my_st = "test"
my_l1 = [10]
my_l2 = [10]
print("Before the function call: ", my_n, my_st, my_l1, my_l2)
fun(my_n, my_st, my_l1, my_l2)
print("After the function call: ", my_n, my_st, my_l1, my_l2)
```

Example

- What happens with the following code?

```
def fun(n, st, l1, l2):
    n = n*2
    st = st.capitalize()
    l1.append(20)
    l2 = [10,20]
    print("Inside the function: ", n, st, l1, l2)

my_n = 5
my_st = "test"
my_l1 = [10]
my_l2 = [10]
print("Before the function call: ", my_n, my_st, my_l1, my_l2)
fun(my_n, my_st, my_l1, my_l2)
print("After the function call: ", my_n, my_st, my_l1, my_l2)
```

```
Before the function call: 5 test [10] [10]
Inside the function: 10 Test [10, 20] [10, 20]
After the function call: 5 test [10, 20] [10]
```

Example

- What happens with the following code?

```
def fun(n, st, l1, l2):
    n = n*2
    st = st.capitalize()
    l1.append(20)
    l2 = [10,20]
    print("Inside the function:", n, st, l1, l2)

fun(my_n, my_st, my_l1, my_l2)
```

LOCAL VARIABLES

Immutable objects

Mutable objects

Example

- What happens with the following code?

```
def fun(n, st, l1, l2):
    n = n*2
    st = st.capitalize()
    l1.append(20)
    l2 = [10,20]
    print("Inside the function:", n, st, l1, l2)

fun(my_n, my_st, my_l1, my_l2)
```

FUNCTION PARAMETER

LOCAL VARIABLES

Immutable objects

Mutable objects

Example

- How can we then maintain changes to parameters outside the function?

Example

- Use return values in the function and re-assign the variables outside the function.

```
def fun(n, st, l1, l2):
    n = n*2
    st = st.capitalize()
    l1.append(20)
    l2 = [10,20]
    print("Inside the function:", n, st, l1, l2)
    return(st)

my_n = 5
my_st = "test"
my_l1 = [10]
my_l2 = [10]
print("Before the function call: ", my_n, my_st, my_l1, my_l2)
my_st = fun(my_n, my_st, my_l1, my_l2)
print("After the function call: ", my_n, my_st, my_l1, my_l2)
```

```
Before the function call: 5 test [10] [10]
Inside the function: 10 Test [10, 20] [10, 20]
After the function call: 5 Test [10, 20] [10]
```

Exercise (6.1)

- Write a function that takes as input a number and returns True if it is a prime number, and False otherwise, by using a while loop.
- Write also the main program to use the function.

Exercise (6.2)

- Write a function that computes the Collatz's sequence of a given integer **n** based on the following algorithm:
  - let **n** be a positive integer
  - if **n** = 1 then stop
    - print **n**
    - If **n** is even, divide it by two (integer division); otherwise multiply it by 3 and then add 1
    - iterate until you reach 1
- Write also the main program to use the function.
- E.g. Collatz(19)
  19 58 29 88 44 22 11 34 17 52 26 13 40 20 10 5 16 8 4 2 1